```
$ curl /users/123 | usecase | repository | db
```

Clean Architecture

//

Controller	Usecase	Repository	Presenter

```
$ cat blueprint.md
```

01 /

02 /

03 /

TypeScript +

04 /

DB——

PART 01

□□□□□

```
// UI □□□□□□
app.get('/users', (req, res) => {
  const users = db.query('SELECT * FROM users');
  res.json(users.map(u => ({
    name: u.name,
    email: u.email,
    createdAt: u.created_at.format('YYYY-MM-DD')
  })));
});
```

INCOMING REQUESTS

□□□□□□□□□ GraphQL□□

→ □□□□□□□□□□□□□□

□□□□□□□□□□

→ □ 10 □□□

□□□□□ MongoDB□□

→ SQL □□□□□□□□□□

[illegible]

[illegible]
[illegible] Repository

[illegible] API [illegible]
[illegible] Presenter

[illegible]
[illegible] Controller

INVARIANT

[illegible]Usecase[illegible] —— [illegible]

PART 02

[illegible]

Controller = [illegible]

[illegible]

Usecase = [illegible]

[illegible]——[illegible] JSON [illegible] XML

Repository = [illegible]

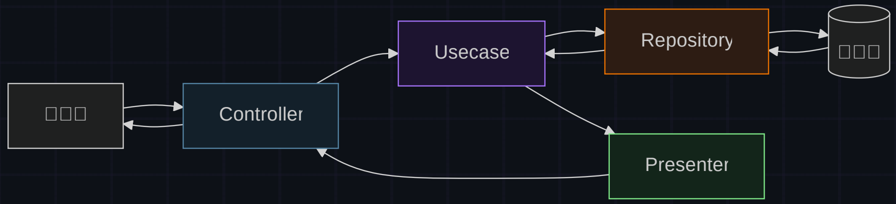
[illegible] MySQL[illegible]MongoDB [illegible] API [illegible]

Presenter = [illegible]

JSON [illegible] XML [illegible]

[illegible]

□□□□□□□□□□□



□□
UI → Usecase → Repository → DB

□□
DB → Repository → Usecase → Presenter → UI□□□□□□□

PART 03

TypeScript

Repository□□□□□

```typescript
export interface UserRepository {
  getUser(id: number): Promise<User | null>;
}

// □□□□ MySQL —— Usecase □□□□□□
export class MySQLUserRepository
  implements UserRepository {
  async getUser(id: number) {
    const rows = await db.query(
      'SELECT * FROM users WHERE id = ?', [id]);
    return rows[0] ?? null;
  }
}
```

Usecase□□□□□□□

```typescript
export class GetUserUseCase {
  constructor(
    private userRepository: UserRepository
  ) {}

  async execute(id: number) {
    const user =
      await this.userRepository.getUser(id);
    if (!user) return null;

    // □□□□□□□□30 □□□□□□□□
    if (user.createdAt < thirtyDaysAgo()) {
      return null;
    }
    return user;
  }
}
```

□□□□□□□□□□

□□□□□ UserRepository —— MySQL □□ Mongo□□□□□□□□□□

Presenter □ Controller

```typescript
// Presenter□□□□□□□
export class UserPresenter {
  static toResponse(user: User | null) {
    if (!user) return { error: 'User not found' };
    return {
      id: user.id,
      name: user.name,
      created_at: user.createdAt.toISOString(),
    };
  }
}
```

□□□□□□□□

```typescript
// routes/user.ts —— □□□□□□□□
const repo = new MySQLUserRepository();
const usecase = new GetUserUseCase(repo);
const controller = new UserController(usecase);

router.get('/users/:id', controller.getUser);
```

COMPOSITION ROOT

Controller □□□□□□□□□□□□Usecase □□□□□□□□□ —— □□□□□□□□□□□

PART 04

MongoDB

MongoDBUserRepository new

XML

Presenter toResponse()

CacheUserRepository MySQLUserRepository DB Usecase

$ —

CHECKPOINT_01

[illegible]Cache[illegible]

- **A** Usecase[illegible]
- **B** Controller[illegible]
- **C** Repository[illegible] CacheUserRepository[illegible] DB

FAQ [illegible]

[illegible] MVC [illegible]

MVC [illegible]Clean Architecture [illegible]

[illegible]

[illegible] 10 [illegible] 100 [illegible] 1 [illegible] 100 [illegible]

[illegible]

[illegible] 2 [illegible]

Repository [illegible]

[illegible] Usecase [illegible]——[illegible]

——

DB——

——

Layers separated.

□□□□Spring Boot × DDD□□□□□□□□□□□□□□

```
$ curl /users/123
controller → usecase → repository → presenter
```